ENSA Béni Mellal
Filière : IACS1

Rapport de TP

Architecture Orientée Services (SOA)

Réalisé par :
BOUHAMIDI-ALAOUI Khaoula

Encadrant :
Pr. BE ELBAGHAZAOUI

Année universitaire 2025/2026

Table des matières

1 Introduction 3
1.1 Objectifs pédagogiques ... 3
1.2 Prérequis ... 3

2 Architecture du système 3

3 Étape 1 : Créer le serveur Eureka 4
3.1 Création du projet ... 4
3.2 Configuration ... 5
3.3 Classe principale ... 5
3.4 Démarrage ... 6

4 Étape 2 : Créer le service Product 7
4.1 Création du projet ... 7
4.2 Configuration ... 8
4.3 Modèle de données ... 9
4.4 Repository ... 9
4.5 Service ... 10
4.6 Contrôleur REST ... 11
4.7 Activation Eureka Client ... 11

5 Étape 3 : Créer le service Order 12
5.1 Création du projet ... 12
5.2 Configuration ... 13
5.3 Modèle de données ... 13
5.4 Client Feign ... 14
5.5 DTO Product ... 15
5.6 Repository ... 16
5.7 Service ... 16
5.8 Controller ... 17

6 Étape 4 : Tester l'application 18
6.1 Démarrage des services ... 18
6.2 Tests avec Postman ... 19

7 Exercice 1 : Ajouter une fonctionnalité 22
7.1 Endpoint de mise à jour du stock ... 22
7.2 Modification de Order Service ... 22
7.3 Test stock insuffisant ... 23

8 Exercice 2 : Créer une API Gateway 24
8.1 Création du projet ... 24
8.2 Configuration des routes ... 24
8.3 Tests via la Gateway ... 25

9 Exercice 3 : Gestion des erreurs 26
9.1 Produit introuvable 26
9.2 Stock insuffisant 26
9.3 Service Product indisponible 27

10 Conclusion 28

1 Introduction

L'architecture orientée services (SOA - Service-Oriented Architecture) est un paradigme de conception logicielle qui permet de structurer une application en services indépendants et réutilisables. Dans ce TP, nous avons réalisé une application e-commerce simple avec Spring Boot en suivant une architecture SOA.

Commentaire

Ce TP nous a permis de comprendre comment découper une application en plusieurs services spécialisés, capables de communiquer entre eux tout en restant indépendants.

1.1 Objectifs pédagogiques

Les objectifs pédagogiques de ce TP sont les suivants :

- Comprendre les principes de l'architecture SOA
- Créer des microservices avec Spring Boot
- Implémenter la communication inter-services via REST
- Gérer une base de données avec Spring Data JPA
- Utiliser un service de découverte (Eureka)

Commentaire

Ces objectifs couvrent les notions fondamentales des applications distribuées modernes : modularité, communication, découverte de services et centralisation des accès.

1.2 Prérequis

Avant de commencer ce TP, nous avons utilisé les outils suivants :

- Java JDK 17 ou supérieur
- Maven 3.6+
- IDE (IntelliJ IDEA, Eclipse ou VS Code)
- Postman pour tester les APIs
- Connaissances de base en Java et Spring

Commentaire

Ces outils étaient indispensables pour créer, exécuter et tester les microservices dans de bonnes conditions.

2 Architecture du système

Notre application e-commerce est composée des quatre microservices suivants :

- Product Service : gestion du catalogue de produits

– Order Service : gestion des commandes clients
– Eureka Server : service de découverte et registre des microservices
– API Gateway : point d'entrée unique pour les requêtes clients

Chaque service est indépendant et utilise sa propre base de données H2 en mémoire pour simplifier les tests.

Commentaire

Cette architecture permet de séparer clairement les responsabilités entre les services et de mieux organiser l'application.

3 Étape 1 : Créer le serveur Eureka

Le serveur Eureka permet aux microservices de s'enregistrer et de se découvrir mutuellement.

3.1 Création du projet

Le projet Eureka Server a été créé à partir de Spring Initializr avec la dépendance **Eureka Server**.

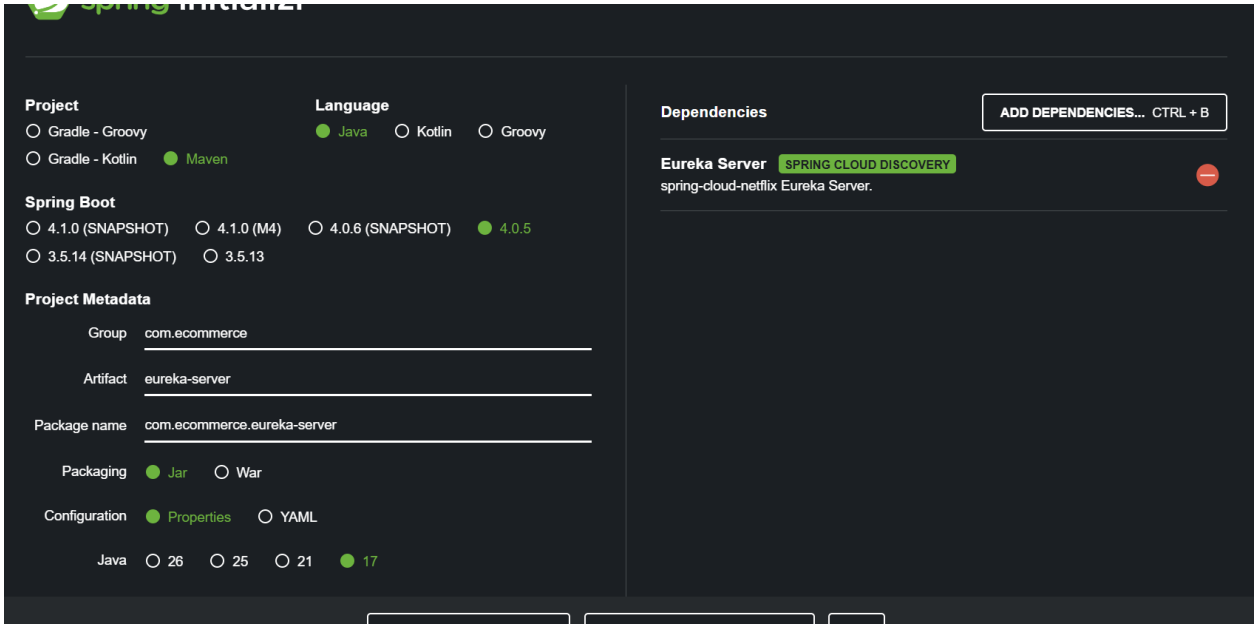


FIGURE 2 – Création du projet Eureka Server

Commentaire

Cette étape constitue la base de l'architecture, car tous les autres services s'appuient sur Eureka pour se retrouver dynamiquement.

3.2 Configuration

Nous avons configuré le fichier application.properties du serveur Eureka.

```
application.properties × | EurekaServerApplication.java | m pom.xml (eure
server.port=8761
spring.application.name=eureka-server

eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

FIGURE 3 – Fichier application.properties de Eureka Server

Commentaire

Cette configuration définit le port 8761 et désactive l'enregistrement du serveur sur lui-même.

3.3 Classe principale

Nous avons ensuite activé Eureka Server dans la classe principale du projet.

```
application.properties | EurekaServerApplication.java × | m pom.xml (eureka-server)
package com.ecommerce.eureka_server;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

FIGURE 4 – Classe principale de Eureka Server

Commentaire

L'annotation d'activation permet à l'application de fonctionner comme serveur de découverte.

3.4 Démarrage

Après compilation et exécution, nous avons vérifié le bon fonctionnement du serveur dans le navigateur.

```
EurekaServerApplication
"C:\Program Files\Eclipse Adoptium\jdk-17.0.18.8-hotspot\bin\java.exe" ...

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.0.5)

2026-04-03T14:22:53.865+01:00  INFO 3988 --- [eureka-server] [           main] c.e.e.EurekaServerApplication            : Starting EurekaServerApplication using Java 17.0.18 w
2026-04-03T14:22:53.869+01:00  INFO 3988 --- [eureka-server] [           main] c.e.e.EurekaServerApplication            : No active profile set, falling back to 1 default prof
2026-04-03T14:22:55.074+01:00  INFO 3988 --- [eureka-server] [           main] o.s.cloud.context.scope.GenericScope     : BeanFactory id=47bdc279-96a3-3147-b5b7-5291dfa67538
2026-04-03T14:22:55.603+01:00  INFO 3988 --- [eureka-server] [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat initialized with port 8761 (http)
2026-04-03T14:22:55.627+01:00  INFO 3988 --- [eureka-server] [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-04-03T14:22:55.628+01:00  INFO 3988 --- [eureka-server] [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/11.0.20]
2026-04-03T14:22:55.732+01:00  INFO 3988 --- [eureka-server] [           main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed
2026-04-03T14:22:56.904+01:00  INFO 3988 --- [eureka-server] [           main] c.n.d.provider.DiscoveryJerseyProvider   : Using JSON encoding codec LegacyJacksonJson
2026-04-03T14:22:56.905+01:00  INFO 3988 --- [eureka-server] [           main] c.n.d.provider.DiscoveryJerseyProvider   : Using JSON decoding codec LegacyJacksonJson
2026-04-03T14:22:57.102+01:00  INFO 3988 --- [eureka-server] [           main] c.n.d.provider.DiscoveryJerseyProvider   : Using XML encoding codec XStreamXml
2026-04-03T14:22:57.102+01:00  INFO 3988 --- [eureka-server] [           main] c.n.d.provider.DiscoveryJerseyProvider   : Using XML decoding codec XStreamXml
2026-04-03T14:22:57.343+01:00  INFO 3988 --- [eureka-server] [           main] o.s.v.b.OptionalValidatorFactoryBean     : Failed to set up a Bean Validation provider: jakarta.
2026-04-03T14:22:58.615+01:00  WARN 3988 --- [eureka-server] [           main] iguration$LoadBalancerCaffeineWarnLogger : Spring Cloud LoadBalancer is currently working with t
2026-04-03T14:22:58.633+01:00  INFO 3988 --- [eureka-server] [           main] o.s.c.n.eureka.InstanceInfoFactory       : Setting initial instance status as: STARTING
2026-04-03T14:22:58.653+01:00  INFO 3988 --- [eureka-server] [           main] com.netflix.discovery.DiscoveryClient    : Initializing Eureka in region us-east-1
2026-04-03T14:22:58.653+01:00  INFO 3988 --- [eureka-server] [           main] com.netflix.discovery.DiscoveryClient    : Client configured to neither register nor query for d
2026-04-03T14:22:58.655+01:00  INFO 3988 --- [eureka-server] [           main] com.netflix.discovery.DiscoveryClient    : Discovery Client initialized at timestamp 17752225786
2026-04-03T14:22:58.688+01:00  INFO 3988 --- [eureka-server] [           main] c.n.eureka.DefaultEurekaServerContext    : Initializing ...
```

```
2026-04-03T14:22:58.615+01:00  WARN 3988 --- [eureka-server] [           main] iguration$LoadBalancerCaffeineWarnLogger : Spring Cloud LoadBalancer is currently working with t
2026-04-03T14:22:58.633+01:00  INFO 3988 --- [eureka-server] [           main] o.s.c.n.eureka.InstanceInfoFactory       : Setting initial instance status as: STARTING
2026-04-03T14:22:58.653+01:00  INFO 3988 --- [eureka-server] [           main] com.netflix.discovery.DiscoveryClient    : Initializing Eureka in region us-east-1
2026-04-03T14:22:58.653+01:00  INFO 3988 --- [eureka-server] [           main] com.netflix.discovery.DiscoveryClient    : Client configured to neither register nor query for d
2026-04-03T14:22:58.655+01:00  INFO 3988 --- [eureka-server] [           main] com.netflix.discovery.DiscoveryClient    : Discovery Client initialized at timestamp 17752225786
2026-04-03T14:22:58.688+01:00  INFO 3988 --- [eureka-server] [           main] c.n.eureka.DefaultEurekaServerContext    : Initializing ...
2026-04-03T14:22:58.690+01:00  INFO 3988 --- [eureka-server] [           main] c.n.eureka.cluster.PeerEurekaNodes       : Adding new peer nodes [http://localhost:8761/eureka/]
2026-04-03T14:22:58.786+01:00  INFO 3988 --- [eureka-server] [           main] c.n.d.provider.DiscoveryJerseyProvider   : Using JSON encoding codec LegacyJacksonJson
2026-04-03T14:22:58.786+01:00  INFO 3988 --- [eureka-server] [           main] c.n.d.provider.DiscoveryJerseyProvider   : Using JSON decoding codec LegacyJacksonJson
2026-04-03T14:22:58.787+01:00  INFO 3988 --- [eureka-server] [           main] c.n.d.provider.DiscoveryJerseyProvider   : Using XML encoding codec XStreamXml
2026-04-03T14:22:58.787+01:00  INFO 3988 --- [eureka-server] [           main] c.n.d.provider.DiscoveryJerseyProvider   : Using XML decoding codec XStreamXml
2026-04-03T14:22:58.825+01:00  INFO 3988 --- [eureka-server] [           main] c.n.eureka.cluster.PeerEurekaNodes       : Replica node URL:  http://localhost:8761/eureka/
2026-04-03T14:22:58.835+01:00  INFO 3988 --- [eureka-server] [           main] c.n.e.registry.AbstractInstanceRegistry  : Finished initializing remote region registries. All
2026-04-03T14:22:58.835+01:00  INFO 3988 --- [eureka-server] [           main] c.n.eureka.DefaultEurekaServerContext    : Initialized
2026-04-03T14:22:58.846+01:00  INFO 3988 --- [eureka-server] [           main] o.s.b.a.e.web.EndpointLinksResolver      : Exposing 1 endpoint beneath base path '/actuator'
2026-04-03T14:22:58.897+01:00  INFO 3988 --- [eureka-server] [           main] o.s.c.n.e.s.EurekaServiceRegistry        : Registering application EUREKA-SERVER with eureka wit
2026-04-03T14:22:58.911+01:00  INFO 3988 --- [eureka-server] [       Thread-9] o.s.c.n.e.server.EurekaServerBootstrap   : isAws returned false
2026-04-03T14:22:58.912+01:00  INFO 3988 --- [eureka-server] [       Thread-9] o.s.c.n.e.server.EurekaServerBootstrap   : Initialized server context
2026-04-03T14:22:58.912+01:00  INFO 3988 --- [eureka-server] [       Thread-9] c.n.e.r.PeerAwareInstanceRegistryImpl    : Got 1 instances from neighboring DS node
2026-04-03T14:22:58.912+01:00  INFO 3988 --- [eureka-server] [       Thread-9] c.n.e.r.PeerAwareInstanceRegistryImpl    : Renew threshold is: 1
2026-04-03T14:22:58.912+01:00  INFO 3988 --- [eureka-server] [       Thread-9] c.n.e.r.PeerAwareInstanceRegistryImpl    : Changing status to UP
2026-04-03T14:22:58.915+01:00  INFO 3988 --- [eureka-server] [       Thread-9] e.s.EurekaServerInitializerConfiguration : Started Eureka Server
2026-04-03T14:22:58.922+01:00  INFO 3988 --- [eureka-server] [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8761 (http) with context path
2026-04-03T14:22:58.923+01:00  INFO 3988 --- [eureka-server] [           main] .s.c.n.e.s.EurekaAutoServiceRegistration : Updating port to 8761
2026-04-03T14:22:59.198+01:00  INFO 3988 --- [eureka-server] [           main] c.e.e.EurekaServerApplication            : Started EurekaServerApplication in 6.454 seconds (pro
2026-04-03T14:23:58.913+01:00  INFO 3988 --- [eureka-server] [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry  : Running the evict task with compensationTime 0ms
2026-04-03T14:24:58.921+01:00  INFO 3988 --- [eureka-server] [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry  : Running the evict task with compensationTime 7ms
2026-04-03T14:25:58.922+01:00  INFO 3988 --- [eureka-server] [a-EvictionTimer] c.n.e.registry.AbstractInstanceRegistry  : Running the evict task with compensationTime 0ms
```

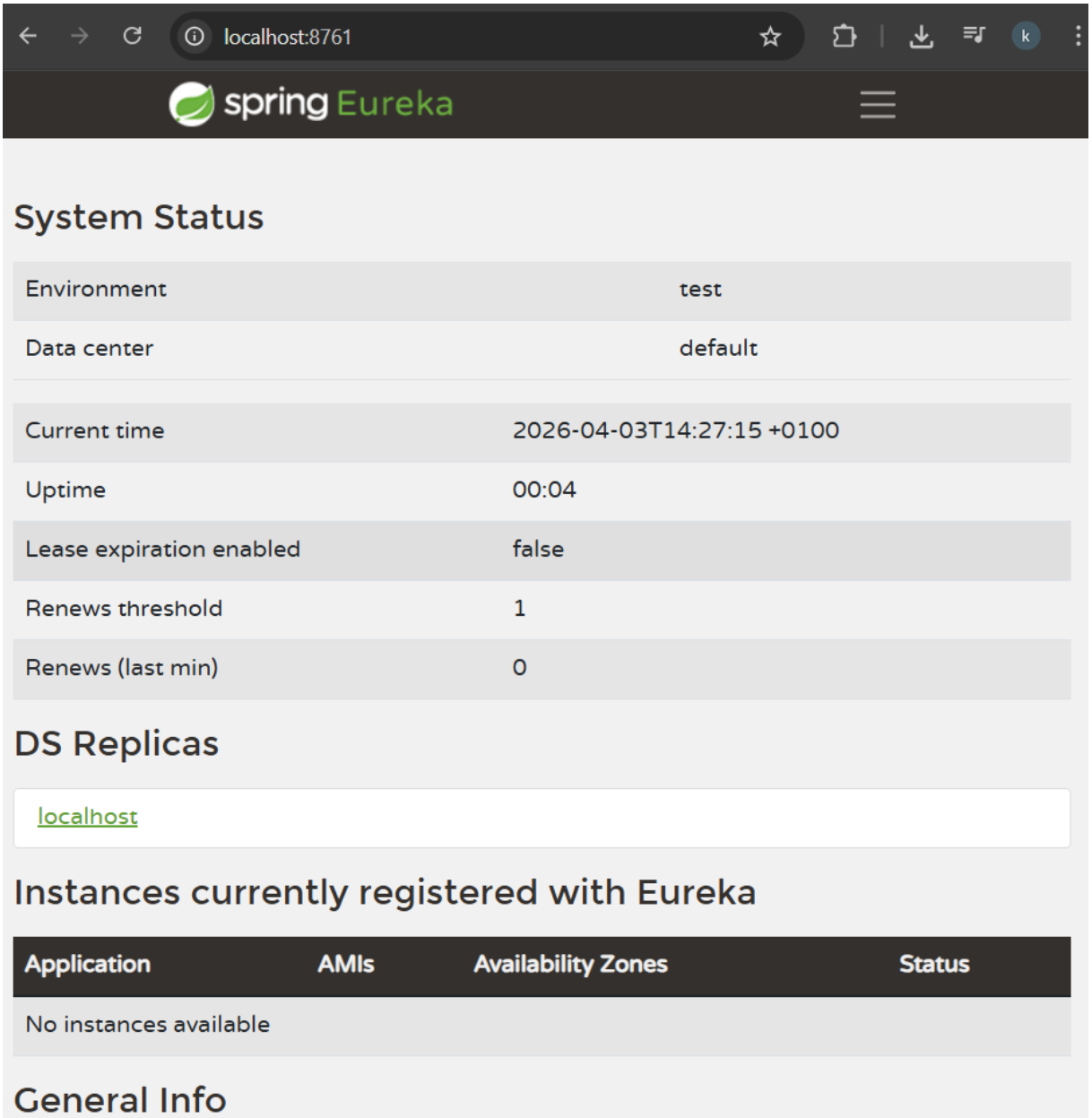


FIGURE 5 – Tableau de bord Eureka

Commentaire

Le tableau de bord Eureka permet de voir les services enregistrés et de vérifier l'état de l'écosystème.

4 Étape 2 : Créer le service Product

Le Product Service permet de gérer les produits du catalogue.

4.1 Création du projet

Le projet a été créé avec les dépendances nécessaires : Spring Web, Spring Data JPA, H2 Database, Eureka Discovery Client et Lombok.

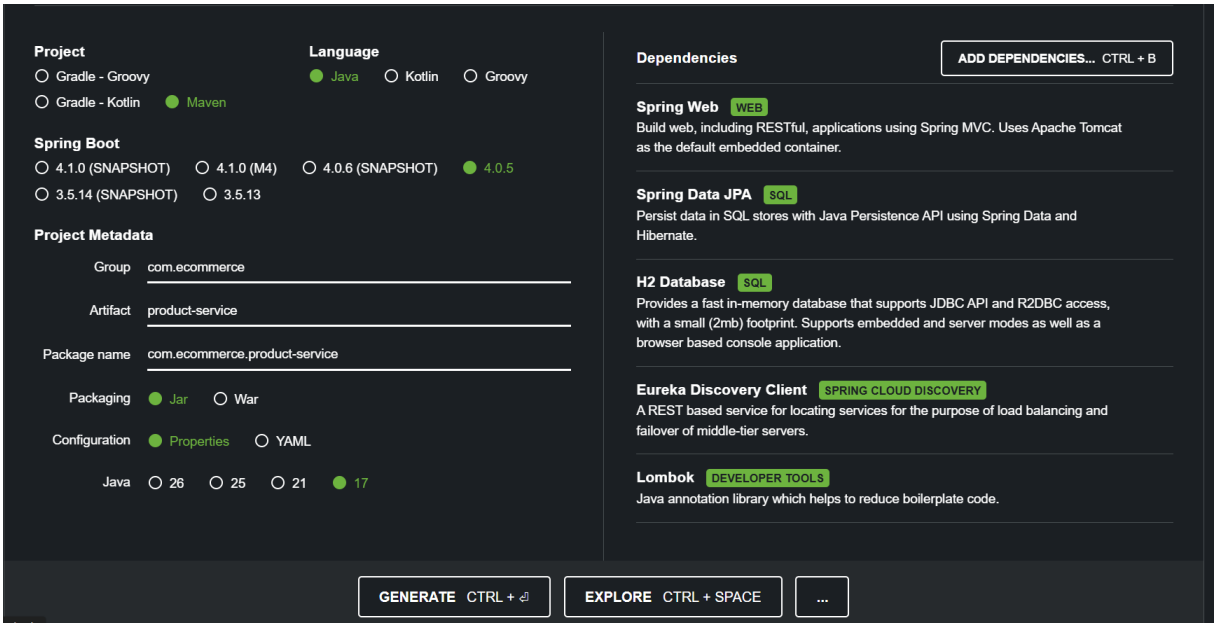


FIGURE 6 – Création du projet Product Service

Commentaire

Le choix de ces dépendances permet de construire un service REST complet, persistant et enregistrable dans Eureka.

4.2 Configuration

Nous avons configuré le fichier application.properties du Product Service.

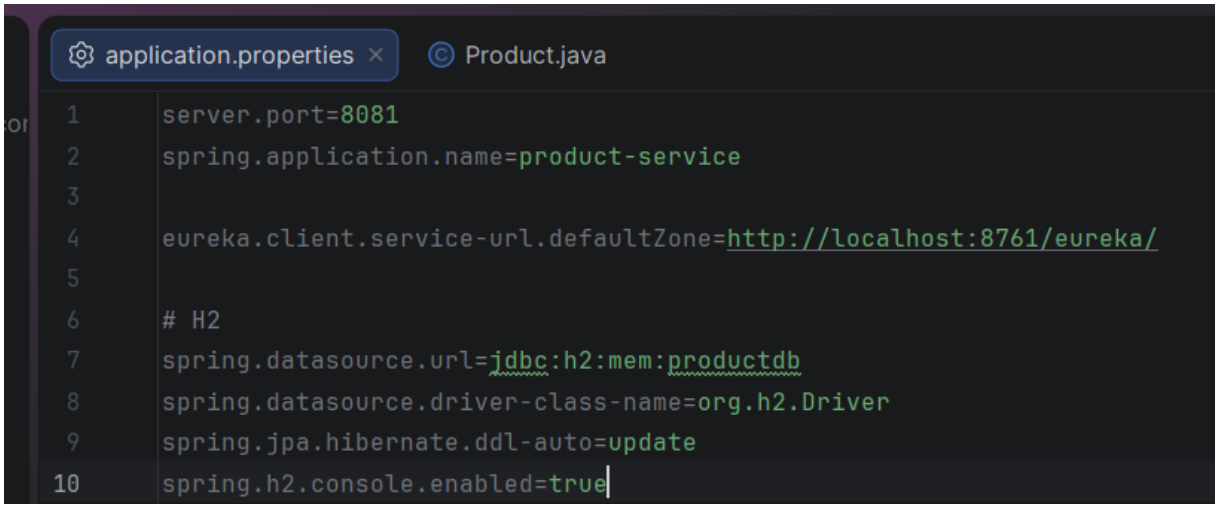


FIGURE 7 – Configuration du Product Service

Commentaire

Cette configuration définit le port 8081, la connexion à H2 et l'adresse du serveur Eureka.

4.3 Modèle de données

Nous avons créé la classe Product.java représentant un produit.

```
application.properties    Product.java ×

package com.ecommerce.product_service.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

FIGURE 8 – Classe Product.java

Commentaire

Cette classe modélise les informations principales d'un produit : nom, description, prix et stock.

4.4 Repository

Nous avons créé l'interface ProductRepository.java.

```
application.properties    Product.java    ProductRepository.java ×

package com.ecommerce.product_service.repository;

import com.ecommerce.product_service.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository  no usages
public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

FIGURE 9 – Interface ProductRepository.java

Commentaire

Le repository permet l'accès aux données sans écrire de requêtes SQL manuelles.

4.5 Service

Nous avons créé la classe ProductService.java contenant la logique métier liée aux produits.

application.properties | Product.java | ProductRepository.java | ProductService.java

```java
package com.ecommerce.product_service.service;

import com.ecommerce.product_service.model.Product;
import com.ecommerce.product_service.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
@RequiredArgsConstructor
public class ProductService {

    private final ProductRepository productRepository;

    public List<Product> getAllProducts() {  no usages
        return productRepository.findAll();
    }
```

```java
    public List<Product> getAllProducts() {  no usages
        return productRepository.findAll();
    }

    public Optional<Product> getProductById(Long id) {  no usages
        return productRepository.findById(id);
    }

    public Product createProduct(Product product) {  no usages
        return productRepository.save(product);
    }

    public void deleteProduct(Long id) {  no usages
        productRepository.deleteById(id);
    }
}
```

FIGURE 10 – Classe ProductService.java

Commentaire

La couche service centralise les opérations métier comme l'ajout, la consultation et la suppression.

4.6 Contrôleur REST

Nous avons créé la classe ProductController.java pour exposer les endpoints REST.

```
application.properties   ProductController.java   Product.java   ProductRepository.java   Proc
```

```java
package com.ecommerce.product_service.controller;

import com.ecommerce.product_service.model.Product;
import com.ecommerce.product_service.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    @GetMapping
    public List<Product> getAllProducts() { return productService.getAllProducts(); }
```

```java
public class ProductController {
    public List<Product> getAllProducts() { return productService.getAllProducts(); }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        return productService.getProductById(id) Optional<Product>
                .map(ResponseEntity::ok) Optional<ResponseEntity<...>>
                .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public Product createProduct(@RequestBody Product product) { return productService.createProduct(product); }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

FIGURE 11 – Classe ProductController.java

Commentaire

Le contrôleur sert d'interface entre le client HTTP et la logique métier du service.

4.7 Activation Eureka Client

Nous avons activé l'enregistrement du Product Service dans Eureka.

```java
package com.ecommerce.product_service;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

FIGURE 12 – Classe principale du Product Service

Commentaire

Grâce à cette activation, le Product Service peut être découvert dynamiquement par les autres services.

5 Étape 3 : Créer le service Order

Le Order Service permet de gérer les commandes et communique avec Product Service.

5.1 Création du projet

Le projet a été créé avec les dépendances Spring Web, Spring Data JPA, H2 Database, Eureka Discovery Client, Lombok et OpenFeign.

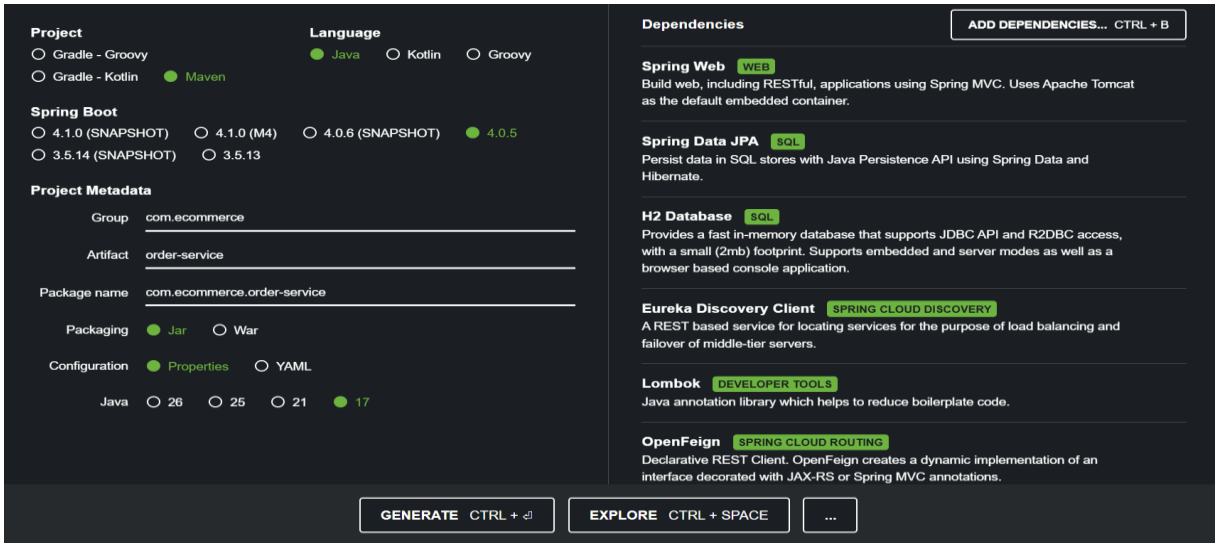


FIGURE 13 – Création du projet Order Service

Commentaire

L'ajout de Feign est essentiel pour permettre la communication entre Order Service et Product Service.

5.2 Configuration

Nous avons configuré le fichier application.properties du Order Service.

```
server.port=8082
spring.application.name=order-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

spring.datasource.url=jdbc:h2:mem:orderdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

FIGURE 14 – Configuration du Order Service

Commentaire

Cette configuration définit le port 8082, la base H2 et la connexion au serveur Eureka.

5.3 Modèle de données

Nous avons créé la classe Order.java représentant une commande.

application.properties OrderServiceApplication.java Order.java

```java
package com.ecommerce.orderservice.model;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long productId;
    private String productName;
    private Integer quantity;
    private Double totalPrice;
    private LocalDateTime orderDate;
    private String status;
}
```

FIGURE 15 – Classe Order.java

Commentaire

Cette classe contient les informations liées à une commande, y compris le produit, la quantité, le prix total et le statut.

5.4 Client Feign

Nous avons créé la classe ProductClient.java pour interroger le Product Service.

```java
package com.ecommerce.orderservice.client;

import com.ecommerce.orderservice.dto.ProductDTO;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(name = "product-service")  no usages
public interface ProductClient {

    @GetMapping("/api/products/{id}")  no usages
    ProductDTO getProductById(@PathVariable Long id);
}
```

FIGURE 16 – Classe ProductClient.java

Commentaire

Le client Feign simplifie les appels REST inter-services en évitant l'écriture manuelle des requêtes HTTP.

5.5 DTO Product

Nous avons créé la classe ProductDTO.java utilisée pour transférer les données du produit.

```java
package com.ecommerce.orderservice.dto;

import lombok.*;

@Data  2 usages
@NoArgsConstructor
@AllArgsConstructor
public class ProductDTO {

    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

FIGURE 17 – Classe ProductDTO.java

Commentaire

Le DTO permet de transporter proprement les données nécessaires entre services.

5.6 Repository

Nous avons créé OrderRepository.java.

```
operties   ProductDTO.java   OrderRepository.java ×   ProductClient.java
1  package com.ecommerce.orderservice.repository;
2
3  import com.ecommerce.orderservice.model.Order;
4  import org.springframework.data.jpa.repository.JpaRepository;
5  import org.springframework.stereotype.Repository;
6
7  @Repository  no usages
8  public interface OrderRepository extends JpaRepository<Order, Long> {
9  }
```

FIGURE 18 – Interface OrderRepository.java

Commentaire

Ce repository assure l'accès aux commandes dans la base H2.

5.7 Service

Nous avons créé OrderService.java contenant la logique métier de création et consultation des commandes.

```java
package com.ecommerce.orderservice.service;

import com.ecommerce.orderservice.client.ProductClient;
import com.ecommerce.orderservice.dto.ProductDTO;
import com.ecommerce.orderservice.model.Order;
import com.ecommerce.orderservice.repository.OrderRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.List;

@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepository orderRepository;
    private final ProductClient productClient;

    public List<Order> getAllOrders() {  no usages
        return orderRepository.findAll();
    }

    public Order createOrder(Long productId, Integer quantity) {  no usages
        ProductDTO product = productClient.getProductById(productId);

        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");

        return orderRepository.save(order);
    }
}
```

FIGURE 19 – Classe OrderService.java

Commentaire

Cette couche effectue les appels vers Product Service, calcule le prix total et crée la commande.

5.8 Controller

Nous avons créé OrderController.java exposant les endpoints REST des commandes.

```java
package com.ecommerce.orderservice.controller;

import com.ecommerce.orderservice.model.Order;
import com.ecommerce.orderservice.service.OrderService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;

    @GetMapping
    public List<Order> getAllOrders() {
        return orderService.getAllOrders();
    }

    @PostMapping
    public Order createOrder(@RequestParam Long productId, @RequestParam Integer quantity) {
        return orderService.createOrder(productId, quantity);
    }
}
```

FIGURE 20 – Classe OrderController.java

Commentaire

Le contrôleur permet de lister les commandes et d'en créer de nouvelles à partir de paramètres HTTP.

6 Étape 4 : Tester l'application

6.1 Démarrage des services

Pour tester correctement l'application, nous avons démarré les services dans l'ordre suivant :

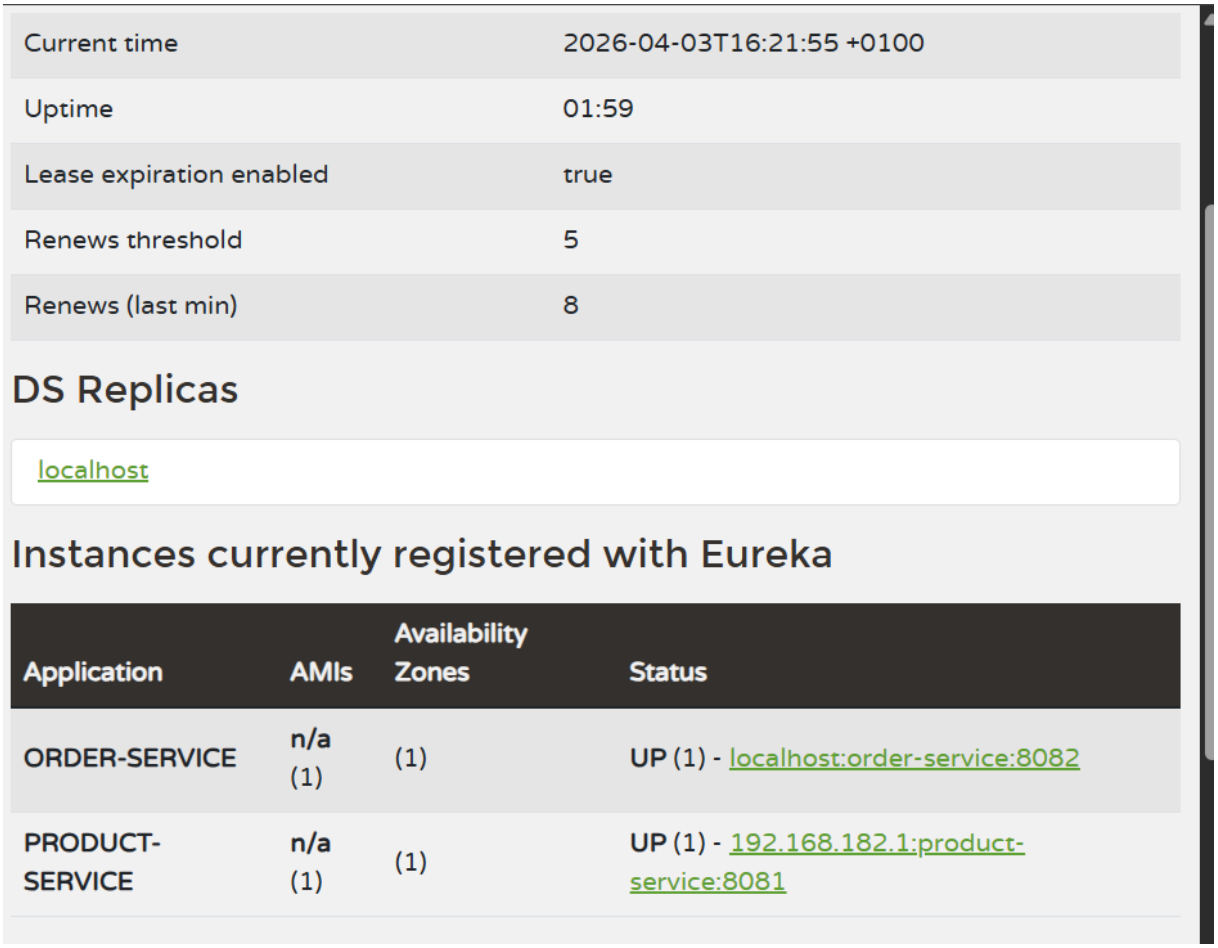
1. Eureka Server (port 8761)
2. Product Service (port 8081)
3. Order Service (port 8082)

Après le démarrage, nous avons ouvert l'interface Eureka à l'adresse :

http://localhost:8761

Nous avons ensuite vérifié que les deux services suivants apparaissaient bien dans le tableau de bord :

— PRODUCT-SERVICE
— ORDER-SERVICE



Current time	2026-04-03T16:21:55 +0100
Uptime	01:59
Lease expiration enabled	true
Renews threshold	5
Renews (last min)	8

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDER-SERVICE	n/a (1)	(1)	UP (1) - localhost:order-service:8082
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.182.1:product-service:8081

FIGURE 22 – Vérification de l'enregistrement des services dans Eureka

Commentaire

Cette étape est essentielle car elle confirme que les micro services sont bien démarrés et enregistrés auprès du serveur Eureka avant les tests Postman.

6.2 Tests avec Postman

Test 1 : Créer un produit

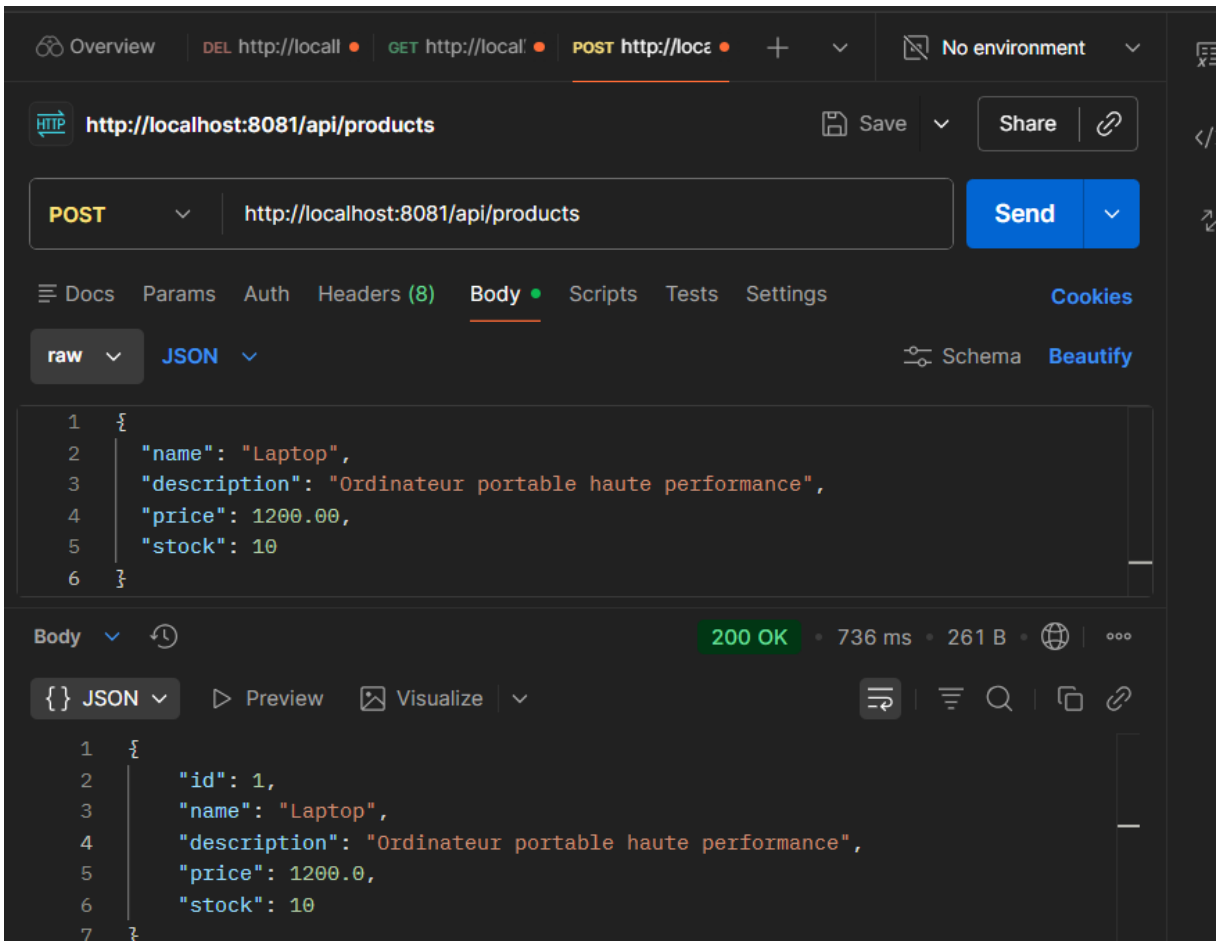


FIGURE 23 – Création d'un produit avec Postman

Test 2 : Lister les produits

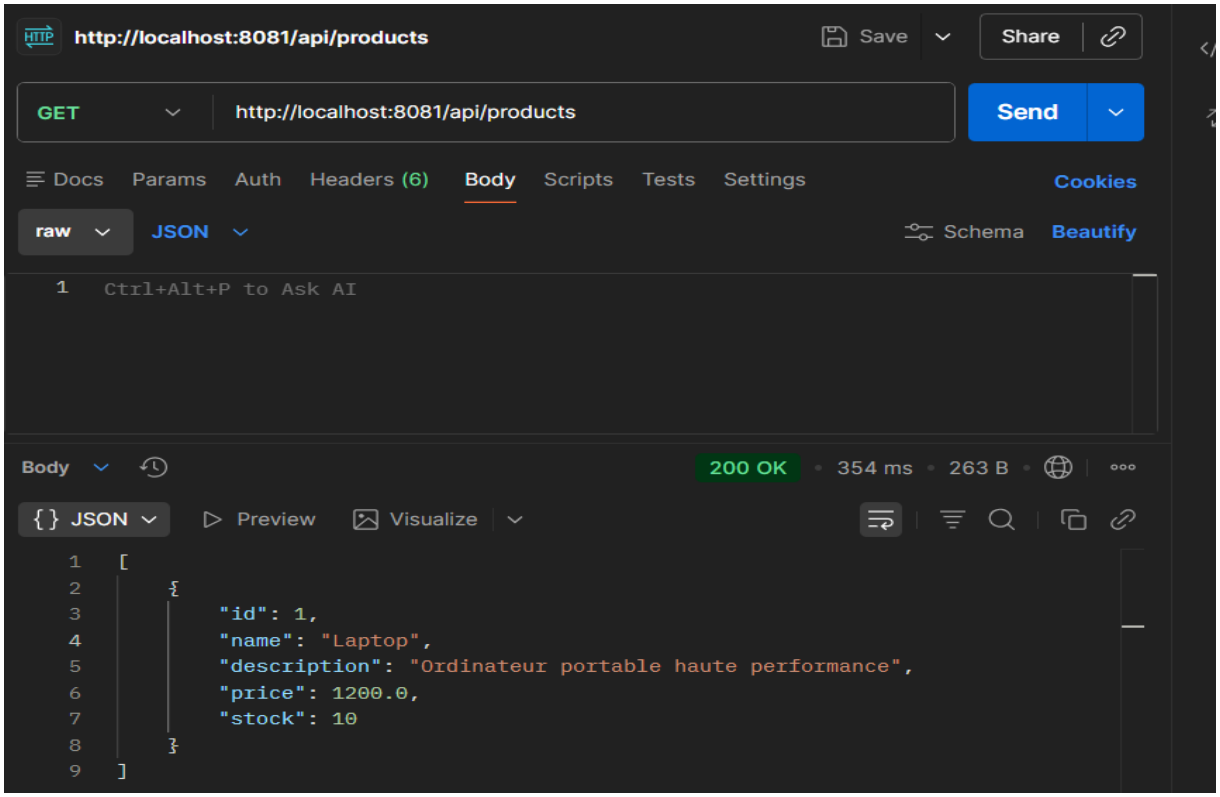


FIGURE 24 – Liste des produits avec Postman

Test 3 : Créer une commande

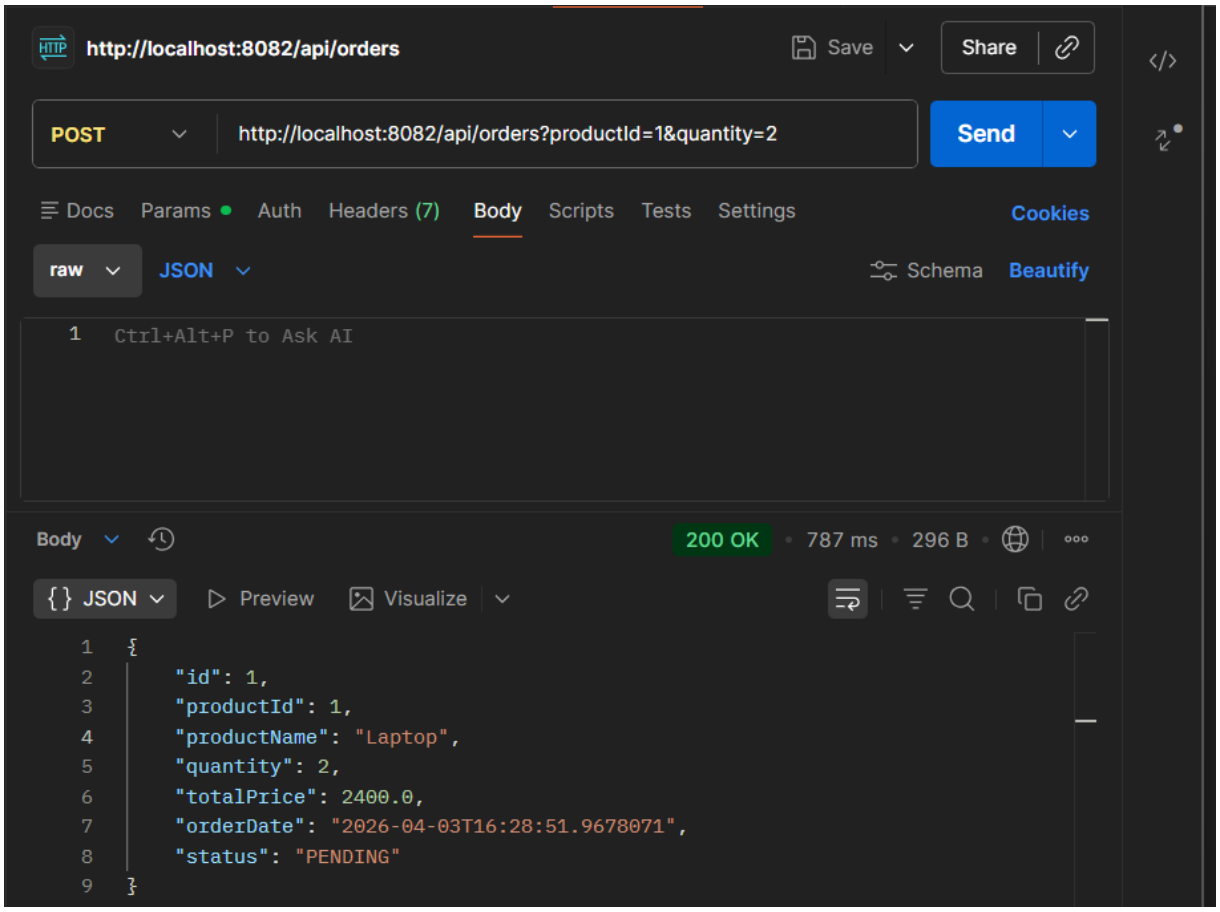


FIGURE 25 – Création d'une commande avec Postman

Test 4 : Lister les commandes

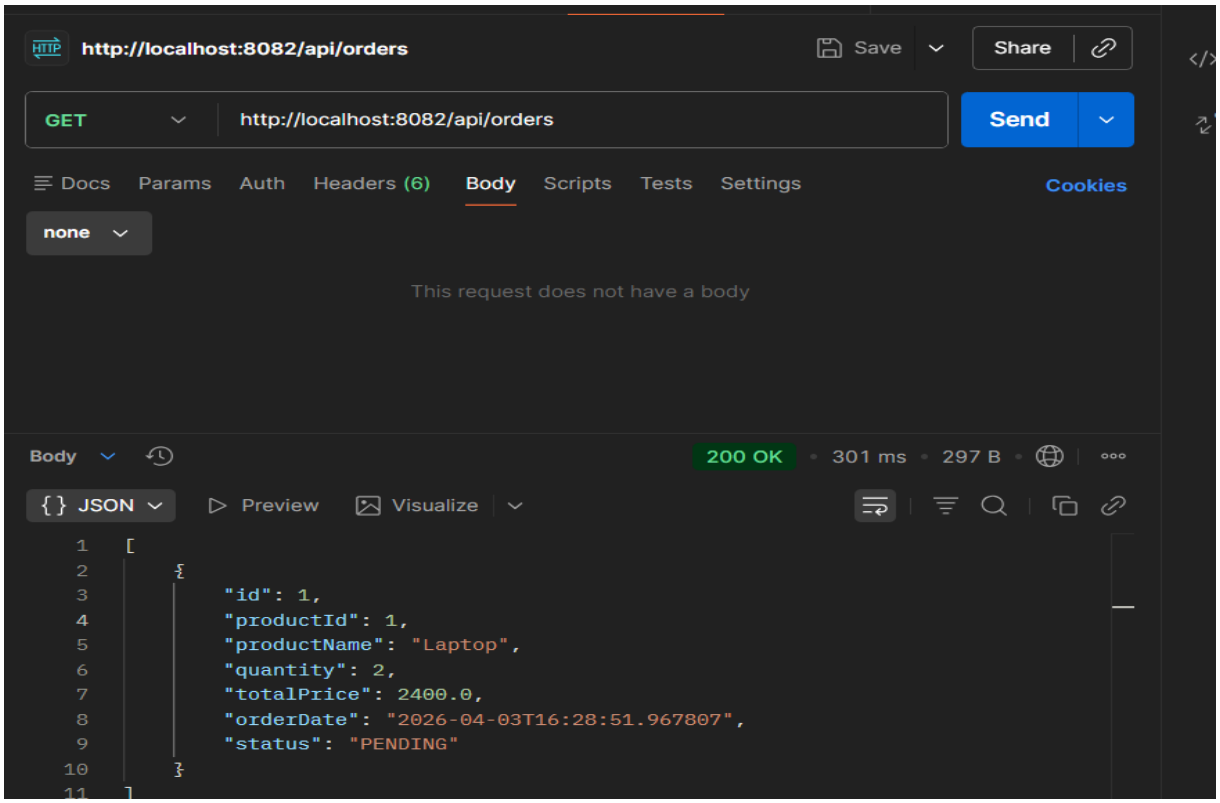


FIGURE 26 – Liste des commandes avec Postman

Commentaire

Ces tests permettent de valider le fonctionnement des endpoints REST et la communication correcte entre les services.

7 Exercice 1 : Ajouter une fonctionnalité

Dans cet exercice, nous avons ajouté une méthode de mise à jour du stock après chaque commande.

7.1 Endpoint de mise à jour du stock

Nous avons ajouté un endpoint PUT /api/products/{id}/stock dans Product Service.

```
public Product updateStock(Long id, Integer quantity) {  1 usage
    Product product = productRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Produit int

    if (quantity == null || quantity <= 0) {
        throw new RuntimeException("Quantité invalide");
    }

    if (product.getStock() < quantity) {
        throw new RuntimeException("Stock insuffisant");
    }

    product.setStock(product.getStock() - quantity);
    return productRepository.save(product);
}
```

FIGURE 27 – Ajout de l'Endpoint de mise à jour du stock

Commentaire

Cet Endpoint permet de diminuer le stock d'un produit après confirmation d'une commande.

7.2 Modification de Order Service

Nous avons modifié Order Service pour appeler automatiquement cet endpoint après création d'une commande.

```java
OrderService.java   GlobalExceptionHandler.java   OrderController.java

public class OrderService {
    public Order createOrder(Long productId, Integer quantity) {
        ProductDTO product;

        try {
            product = productClient.getProductById(productId);
        } catch (FeignException.NotFound e) {
            throw new RuntimeException("Produit introuvable");
        } catch (Exception e) {
            throw new RuntimeException("Service produit indisponible")
        }

        if (product == null) {
            throw new RuntimeException("Produit introuvable");
        }

        if (quantity == null || quantity <= 0) {
            throw new RuntimeException("Quantité invalide");
        }

        if (product.getStock() < quantity) {
            throw new RuntimeException("Stock insuffisant");
        }
```

FIGURE 28 – Modification de Order Service pour mettre à jour le stock

Commentaire

Cette évolution rend le système plus réaliste, car une commande modifie directement le stock du produit concerné.

7.3 Test stock insuffisant

Nous avons également ajouté une gestion du cas où le stock est insuffisant.

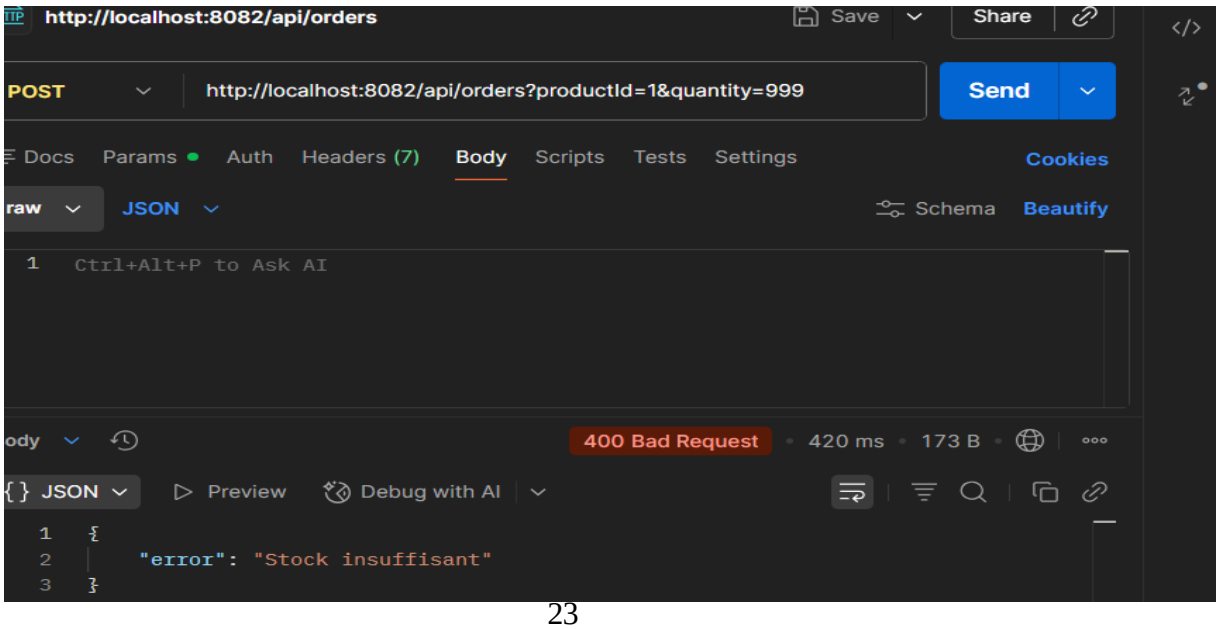


FIGURE 29 – Gestion du stock insuffisant

Commentaire

Cette vérification empêche la validation de commandes impossibles à honorer.

8 Exercice 2 : Créer une API Gateway

Dans cet exercice, nous avons ajouté un point d'entrée unique pour les requêtes clients.

8.1 Création du projet

Le projet API Gateway a été créé avec Spring Cloud Gateway.

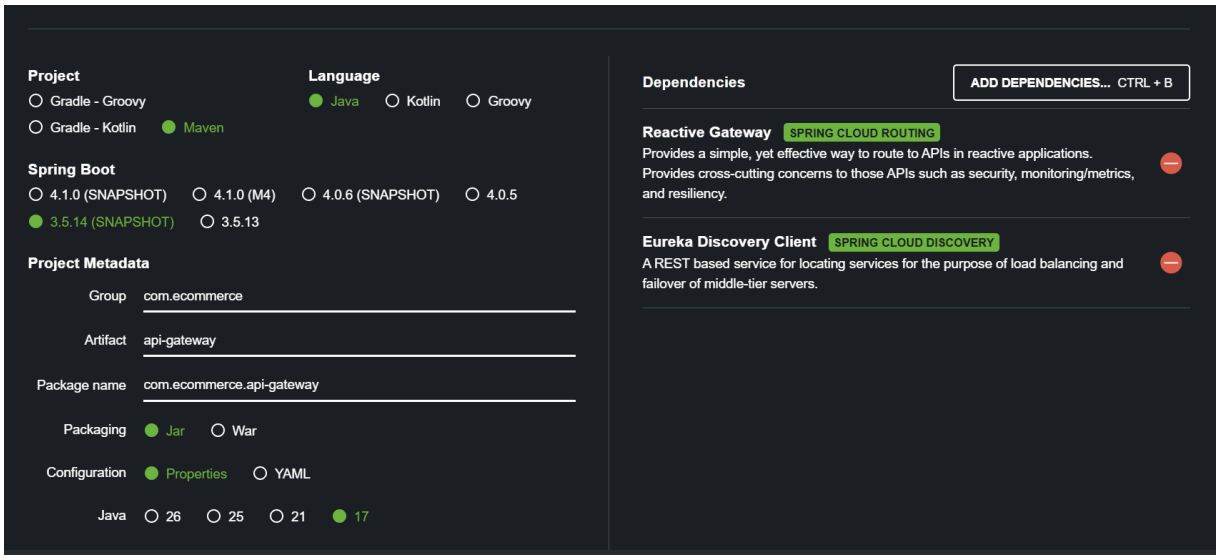


FIGURE 30 – Création du projet API Gateway

Commentaire

La Gateway centralise l'accès à tous les microservices et simplifie l'architecture côté client.

8.2 Configuration des routes

Nous avons configuré les routes vers Product Service et Order Service.

FIGURE 31 – Configuration des routes de l'API Gateway

Commentaire

Cette configuration définit comment la Gateway redirige les requêtes vers les bons services.

8.3 Tests via la Gateway

Nous avons testé l'accès aux services via le port 8080.

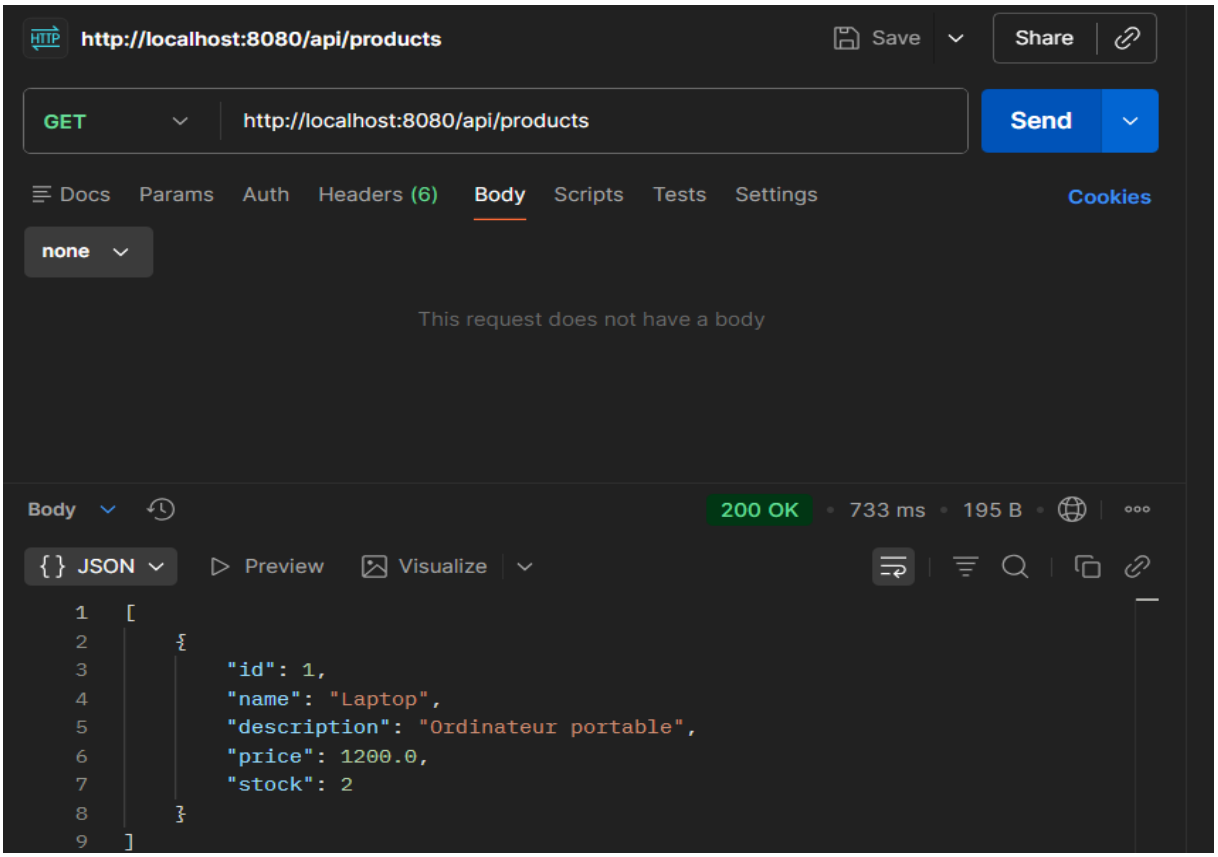


FIGURE 32 – Test Product Service via API Gateway

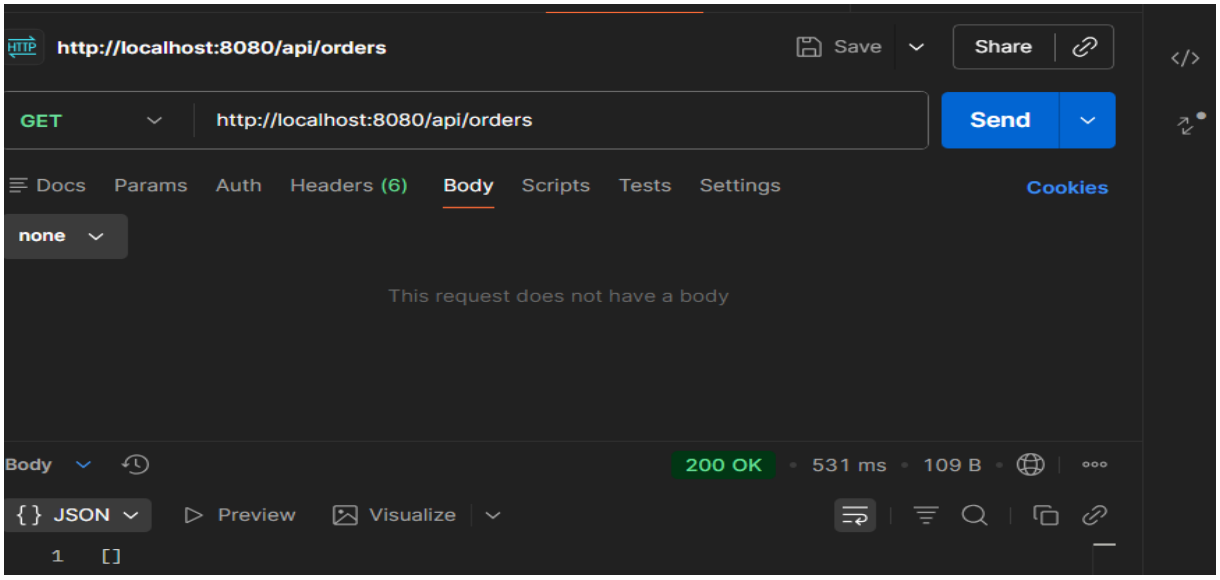


FIGURE 33 – Test Order Service via API Gateway

Commentaire

La Gateway joue le rôle de point d'entrée unique, ce qui simplifie l'appel des services par le client.

9 Exercice 3 : Gestion des erreurs

Dans cet exercice, nous avons amélioré la gestion des erreurs dans Order Service.

9.1 Produit introuvable

Nous avons géré le cas où l'identifiant du produit n'existe pas.

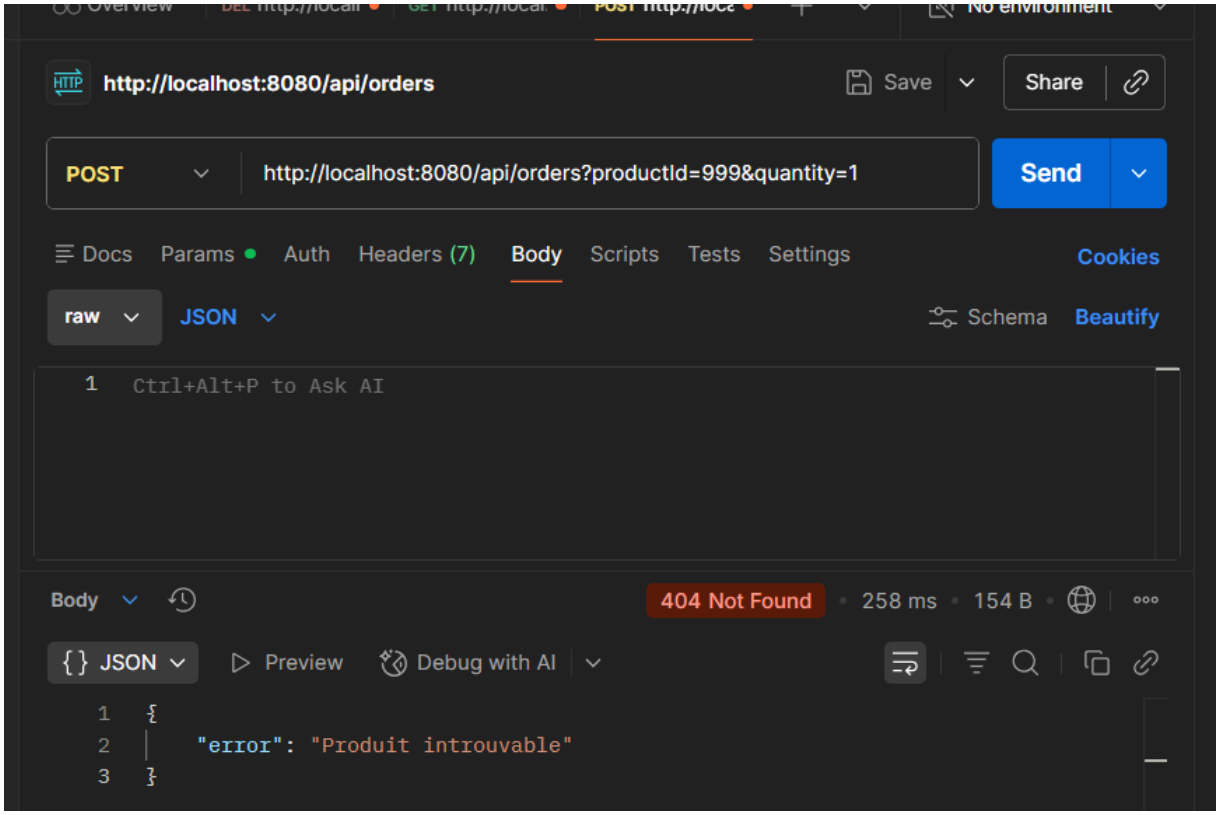


FIGURE 34 – Erreur : produit introuvable

Commentaire

Dans ce cas, l'application retourne une erreur 404 avec un message explicite.

9.2 Stock insuffisant

Nous avons géré le cas où la quantité demandée dépasse le stock disponible.

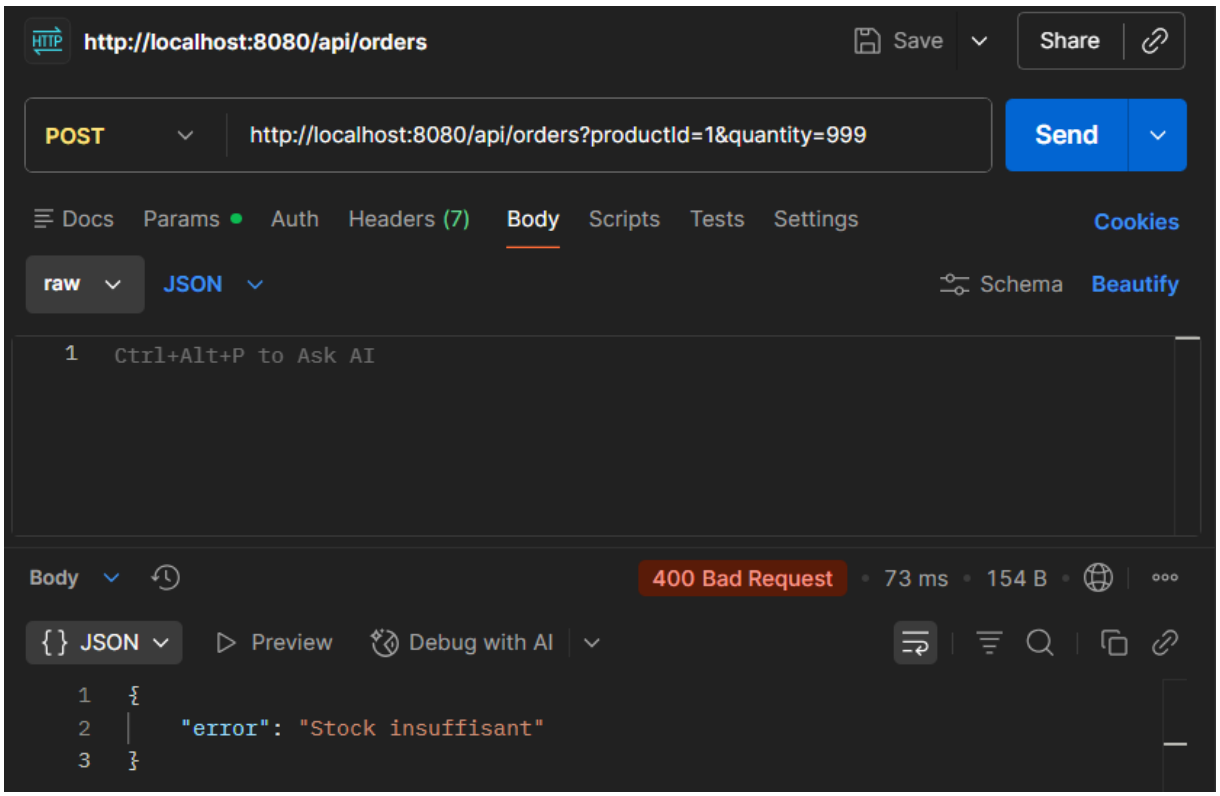


FIGURE 35 – Erreur : stock insuffisant

Commentaire

Le système retourne ici une erreur 400 indiquant que la demande est invalide.

9.3 Service Product indisponible

Nous avons géré le cas où Product Service est arrêté ou inaccessible.

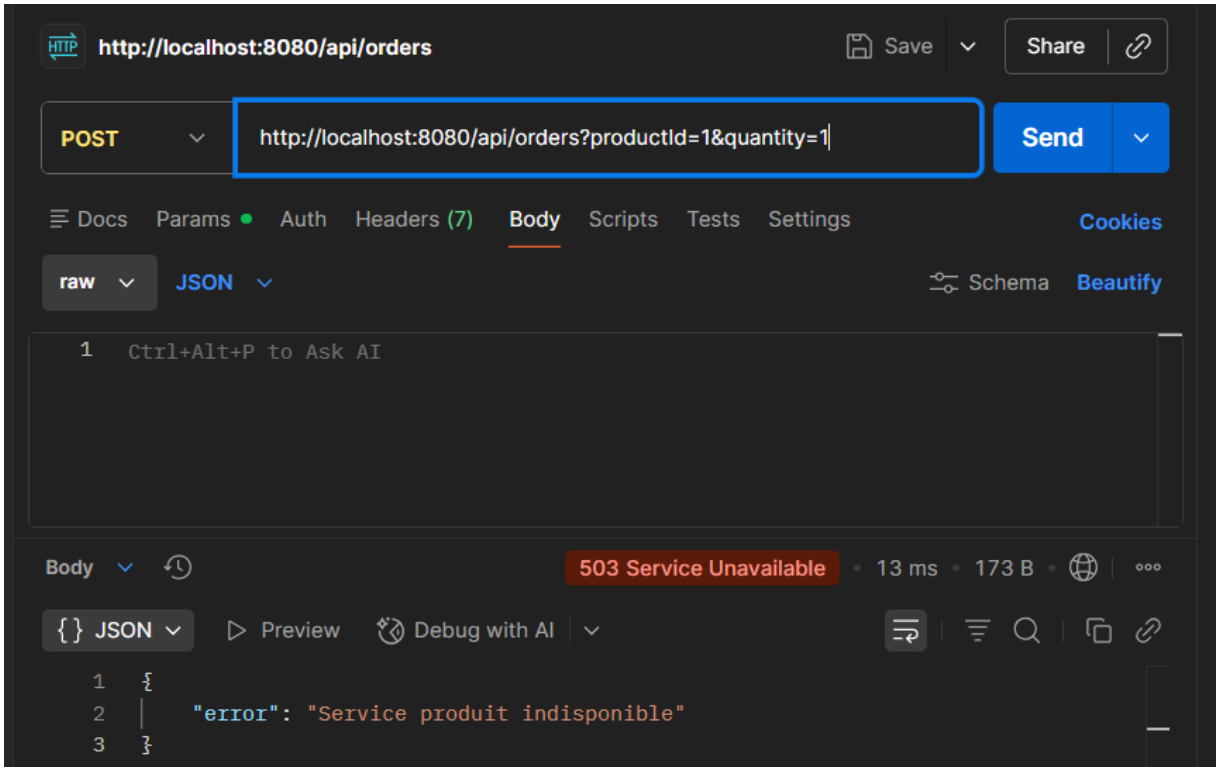


FIGURE 36 – Erreur : service produit indisponible

Commentaire

Cette gestion permet d'informer clairement le client qu'un service distant est momentanément indisponible.

6 Conclusion

Ce TP nous a permis de mettre en œuvre une architecture orientée services complète à l'aide de Spring Boot, Eureka, Feign et API Gateway.

Commentaire

Nous avons appris à concevoir une application distribuée, à faire communiquer les services entre eux, à centraliser les accès et à traiter proprement les erreurs.